

Exam 1, Convex Optimization

Departamento de Matemáticas, Universidad de los Andes
Colombia

Student's name:	Luis Alejandro Rubiano
Student's code:	202013482
Email address:	la.rubiano@uniandes.edu.co
Class:	Convex Optimization
Instructor:	Mauricio Junca

- 3 In a multiclass classification problem with s classes one has data of the form $(a, l) \in \mathbb{R}^n \times \{1, \dots, s\}$, and one wants to find a linear classifier for each class, that is, one wants to find $x_l \in \mathbb{R}^n$ that minimizes the loss function**

$$g(x_1, \dots, x_s; a, l) = \max_{m \neq l} [1 + \langle a, x_m - x_l \rangle]_+$$

where $[t]_+ = \max\{0, t\}$. Note that given (a, l) , it is true that $g = 0$ if and only if the classifiers of each class have a big margin, that is, for each $m \neq l$ one has that

$$\langle a, x_l \rangle \geq \langle a, x_m \rangle + 1.$$

Given a dataset $\{(a_i, l_i)\}_{i=1}^N$ one wants to solve the problem

$$\min \frac{1}{N} \sum_{i=1}^N g(x_1, \dots, x_s; a_i, l_i)$$

subject to $\|x_j\|_2 \leq r, j = 1, \dots, s.$

Consider the database of handwritten digits (train.csv), in this case $n = 784$ and $s = 10$. From the more than 4000 data for each digit, choose $N > 5000$ and take $r = 3$ to solve the problem with each of the following methods:

Here the images were regularized to make them have norm 3 (projection to the sphere of radius 3). The subgradient was calculated in the following way:

- Initialize 10 arrays $\partial x_1, \dots, \partial x_{10}$ with zeros.
- For each (a, l) do:
 - For m from 1 to 10, $m \neq l$ do:
 - * Compute the value of $1 + \langle a, x_m - x_l \rangle$.
 - * If this value is greater than zero, save it
 - Find the maximum of the saved values, and its index m , if multiple values are the maximum, choose one randomly

- Augment the subgradient of x_m by a_i and decrease the subgradient of x_l by a_i , this works because this will be the derivative of one of the active restrictions, and thus it belongs to the subgradient of the objective function.
- Keep the other subgradients as they are.
- This sum will be the subgradient of the objective function, as the subgradient of the sum of functions is the Minkowski sum of the subgradients of each function.

For the stochastic subgradient method, the batch size was 10 and the stochastic subgradient was calculated as before but only for the 10 data in the batch.

All the methods were given 5 minutes to run. And N was chosen to be 10000 with 1000 data for each digit.

a) Projected subgradient with step size $\alpha_k = \frac{\alpha}{\sqrt{k}}$ with $\alpha > 0$. Say how you choose α .

Solution: In this case we used step size $\alpha = 10, 1, 0.1, 0.01$, here is the Python3 code for the projected subgradient method:

```

1 alphas = [10, 1, 0.1, 0.01]
2
3 Fs_for_each = []
4
5 for alpha in alphas:
6     Fs = []
7     start = perf_counter()
8     print(f'Alpha: {alpha}')
9     k = 0
10    X = array([rand(n) for _ in range(s)])
11    for m in range(s):
12        X[m] = r * X[m] / norm(X[m])
13
14    while perf_counter() - start < 300: # 5 minutes
15        a_k = alpha / sqrt(k+1)
16        subgradient_at_X = subgradient(X, images, labels)
17        for m in range(s):
18            X[m] = X[m] - a_k * subgradient_at_X[m]
19        # Project each X[m] to the unit ball
20        for m in range(s):
21            norm_X_m = norm(X[m])
22            if norm_X_m <= r:
23                continue
24            else:
25                X[m] = r * X[m] / norm_X_m
26        if k % 50 == 0:
27            print(f'Iteration {k} completed')
28            print(f'Objective function value: {objective_function(X, images, labels)}')
29        k += 1
30        Fs.append(objective_function(X, images, labels))
31    Fs_for_each.append(Fs)

```

b) Projected subgradient with the α_k given by the Polyak step size. Say how you estimate f^* .

Solution: In this case we used the Polyak step size, the f^* was estimated by taking the value average for each pixel in the 10000 images, and calculating the objective function of that average. Here is the Python3 code for the projected subgradient method with Polyak step size:

```

1 print("\n"*3)
2 f_star = 0.62988 # Optimal value of the objective function from testing
3 Fs_Polyak = []
4 start = perf_counter()
5 k = 0
6 X = array([rand(n) for _ in range(s)])
7 for m in range(s):
8     X[m] = r * X[m] / norm(X[m])
9
10 while perf_counter() - start < 300: # 5 minutes
11     a_k = (objective_function(X, images, labels) - f_star) / (norm(subgradient(X, images,
12     labels))**2)

```

```

12     subgradient_at_X = subgradient(X, images, labels)
13
14     for m in range(s):
15         X[m] = X[m] - a_k * subgradient_at_X[m]
16     # Project each X[m] to the unit ball
17     for m in range(s):
18         norm_X_m = norm(X[m])
19         if norm_X_m <= r:
20             continue
21         else:
22             X[m] = r * X[m] / norm_X_m
23     if k % 50 == 0:
24         print(f'Iteration {k} completed')
25         print(f'Objective function value: {objective_function(X, images, labels)}')
26     k += 1
27     Fs_Polyak.append(objective_function(X, images, labels))

```

c) Stochastic subgradient with step size $\alpha_k = \frac{R}{L\sqrt{k}}$ and batch size $B = 10$. Say how you choose R and L .

Solution: Here R was chosen to be 10, 1, 0.1, 0.01 and the Lipschitz constant was estimated by taking the norm of the stochastic subgradient of the objective function at each iteration. Here is the Python3 code for the stochastic subgradient method:

```

1     print("\n"*3)
2
3     Rs = [10, 1, 0.1, 0.01]
4     Fs_stochastic = []
5
6     for R in Rs:
7         Fs = []
8         start = perf_counter()
9         print(f'R: {R}')
10        k = 0
11        X = array([rand(n) for _ in range(s)])
12        for m in range(s):
13            X[m] = r * X[m] / norm(X[m])
14
15        while perf_counter() - start < 300: # 5 minutes
16            lipschitz_constants = []
17            subgradient_at_X = stochastic_subgradient(X, images, labels)
18            for m in range(s):
19                a_k = R / (sqrt(k+1) * norm(subgradient_at_X[m]))
20                X[m] = X[m] - a_k * subgradient_at_X[m]
21            # Project each X[m] to the unit ball
22            for m in range(s):
23                norm_X_m = norm(X[m])
24                if norm_X_m <= r:
25                    continue
26                else:
27                    X[m] = r * X[m] / norm_X_m
28            if k % 50 == 0:
29                print(f'Iteration {k} completed')
30                print(f'Objective function value: {objective_function(X, images, labels)}')
31            k += 1
32            Fs.append(objective_function(X, images, labels))
33            Fs_stochastic.append(Fs)
34
35        # Save the results in csv
36        df = DataFrame(Fs_for_each)
37        df.to_csv('Fs_for_each.csv')
38
39        df = DataFrame(Fs_Polyak)
40        df.to_csv('Fs_Polyak.csv')
41
42        df = DataFrame(Fs_stochastic)
43        df.to_csv('Fs_stochastic.csv')

```

To compare each of the methods do a graph of f_k^* limiting the number of iterations of each method to the possibles in a time window that is the same for all. Detail all the calculations needed to implement the methods.

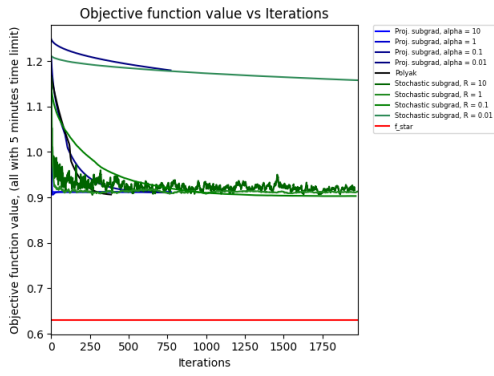


Figure 1: Objective function value vs Iterations

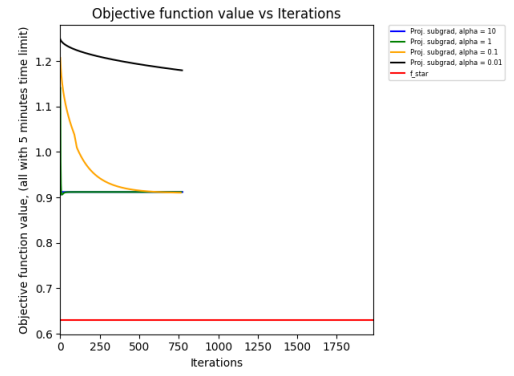


Figure 2: Projected subgradient vs Iterations

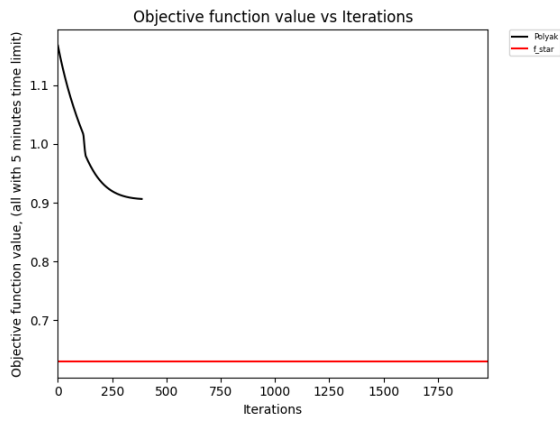


Figure 3: Polyak vs Iterations

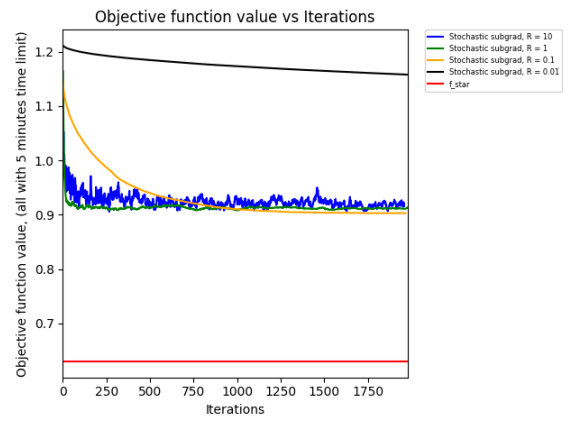


Figure 4: Stochastic subgradient vs Iterations